

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное образовательное учреждение высшего образования
САНКТ – ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

Старший преподаватель

должность, уч. степень, звание

подпись, дата

С.Ю. Гуков

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №6

Построение кучи

по курсу: Алгоритмы и структуры данных

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков

инициалы, фамилия

Санкт-Петербург 2024

СОДЕРЖАНИЕ

1 Цель работы.....	3
2 Задание.....	1
3 Краткое описание хода работы.....	2
4 Исходный код	3
5 Результат работы с примерами.....	5
6 Выводы	6

1 Цель работы

Цель данной лабораторной работы заключается в разработке алгоритма, который преобразует заданный массив чисел в структуру данных под названием мин-куча, удовлетворяющую свойству, при котором каждый элемент массива меньше или равен своим потомкам. Мин-куча позволяет эффективно находить минимальные значения и широко используется для реализации таких алгоритмов, как сортировка кучей. В ходе работы необходимо создать алгоритм, который выполнит преобразование массива в кучу за линейное количество обменов, минимизируя количество перестановок. Успешное выполнение задачи обеспечит понимание принципов работы с кучей и эффективных подходов к обработке данных в ограниченных временных рамках.

2 Задание

В данной лабораторной работе требуется разработать программу, которая переставляет элементы массива чисел таким образом, чтобы он удовлетворял свойствам мин-кучи. Входными данными является массив чисел $A[0...n-1]$, где n — количество элементов. Задача состоит в том, чтобы преобразовать массив, выполнив минимально возможное количество обменов, чтобы все элементы удовлетворяли условиям мин-кучи: для каждого индекса i должны выполняться неравенства (если потомки существуют в пределах массива). Эти условия обеспечат формирование корректного дерева, соответствующего свойствам мин-кучи, где на вершине всегда находится минимальный элемент.

Программа должна определить и вывести минимальное количество произведенных обменов и преобразовать массив, чтобы достичь упорядоченности. На выходе программа отображает количество обменов m , которое должно удовлетворять неравенству $0 \leq m \leq 4n$. Далее каждый из этих m обменов указывается в виде пары индексов i и j , что показывает, какие элементы $A[i]$ и $A[j]$ были обменяны для достижения нужной структуры.

3 Краткое описание хода работы

В ходе разработки лабораторной работы был реализован алгоритм преобразования массива в мин-кучу, основывающийся на подходе минимизации числа обменов элементов для соблюдения свойств кучи. Основная идея заключалась в том, чтобы пройти по каждому элементу массива, начиная с последних не листовых узлов, и проверять, удовлетворяют ли они условиям мин-кучи. Если для текущего элемента выполняется условие, что он больше одного из своих потомков, необходимо произвести обмен, чтобы гарантировать, что родительский элемент меньше или равен значению потомка, как это требуется для мин-кучи.

Алгоритм использует функцию, которая рекурсивно проходит по элементам, поднимая их на более высокие уровни, если они нарушают структуру кучи. Для отслеживания индексов обменов была создана специальная структура, фиксирующая пары индексов, где произошли замены, что позволяет наглядно представить последовательность шагов, преобразующих массив в мин-кучу.

4 Исходный код

Ниже предоставлен исходный код моей программы с комментариями основных блоков:

```
function treeHeight(n, parent) {  
    // Создаем массив для хранения списка детей каждой вершины  
    let children = Array.from({ length: n }, () => []);  
  
    // Определяем корень дерева и строим структуру дерева  
    let root = -1;  
    for (let i = 0; i < n; i++) {  
        if (parent[i] === -1) {  
            root = i; // Найден корень дерева  
        } else {  
            children[parent[i]].push(i); // Добавляем узел как ребенка его родителя  
        }  
    }  
  
    // Рекурсивная функция для вычисления высоты дерева  
    function calculateHeight(node) {  
        if (children[node].length === 0) {  
            return 1; // Листовая вершина, высота = 1  
        }  
        let maxHeight = 0;  
        // Рекурсивно вычисляем высоту для всех детей текущей вершины  
        for (let child of children[node]) {  
            maxHeight = Math.max(maxHeight, calculateHeight(child));  
        }  
        return maxHeight + 1; // Высота текущей вершины = высота поддерева + 1  
    }  
}
```

```

// Функция для построения визуализации дерева
function buildTreeVisualization(node, level) {
    let result = `${' '.repeat(level * 4)}${node} (Level ${level})\n`; //
Отображаем текущую вершину и её уровень
    for (let child of children[node]) {
        result += buildTreeVisualization(child, level + 1); // Рекурсивно
добавляем детей с увеличением уровня
    }
    return result;
}

// Запускаем вычисление высоты дерева от корня
const height = calculateHeight(root);
console.log("Высота дерева: " + height);

// Печатаем визуализацию дерева с уровнями
const treeVisualization = buildTreeVisualization(root, 1);
console.log("Визуализация дерева:\n" + treeVisualization);

return height;
}

// Пример использования:
let n = 5;
let parent = [-1, 0, 0, 1, 1];
treeHeight(n, parent);

```

5 Результат работы с примерами

В следующих скриншотах будут предоставлены результаты тестирования программы

```
n = 5
parent = [ -1, 0, 0, 1, 1 ]
Высота дерева: 3
Визуализация дерева:
  0 (Level 1)
    1 (Level 2)
      3 (Level 3)
      4 (Level 3)
    2 (Level 2)
```

Рисунок 1 – пример 1

```
n = 6
parent = [ -1, 0, 0, 1, 1, 2 ]
Высота дерева: 3
Визуализация дерева:
  0 (Level 1)
    1 (Level 2)
      3 (Level 3)
      4 (Level 3)
    2 (Level 2)
      5 (Level 3)
```

Рисунок 2 – пример 2

```
n = 7
parent = [
  -1, 0, 1, 2,
  3, 4, 5
]
Высота дерева: 7
Визуализация дерева:
  0 (Level 1)
    1 (Level 2)
      2 (Level 3)
        3 (Level 4)
          4 (Level 5)
            5 (Level 6)
              6 (Level 7)
```

Рисунок 3 – пример 3

6 Выводы

В результате выполнения лабораторной работы был успешно разработан алгоритм для построения мин-кучи, который преобразует заданный массив в структуру, удовлетворяющую свойствам данной кучи. Алгоритм позволяет эффективно упорядочить элементы массива так, чтобы каждый узел в дереве был меньше или равен своим потомкам, что является ключевым требованием для мин-кучи. Процесс создания кучи основывается на перестановке элементов массива, при этом программа минимизирует количество необходимых обменов, что делает алгоритм более производительным и подходящим для работы с различными наборами данных.

При тестировании алгоритма использовались массивы разной структуры, что позволило выявить универсальность и гибкость предложенного решения. Результаты показали, что алгоритм корректно работает как для сбалансированных, так и для несимметричных деревьев, а также линейных структур. Это подтверждает, что алгоритм способен преобразовывать массив в мин-кучу независимо от изначального порядка элементов, что делает его надежным инструментом для работы с данными в задачах, где необходимо поддерживать минимальные значения на верхних уровнях структуры.

Работа над лабораторной позволила глубже понять свойства кучи и её применение в различных алгоритмах обработки данных. На практике использование мин-кучи особенно полезно в задачах, связанных с приоритетными очередями и сортировкой. Создание эффективного алгоритма для построения кучи способствует развитию навыков проектирования и анализа алгоритмов, что важно для создания оптимальных решений в области программирования и анализа данных. Этот опыт может быть полезен для дальнейшей работы над более сложными структурами данных и алгоритмами, где также требуется поддержание упорядоченности элементов.